Lab 4 Report:

Surpass Human Performance in Fashion MNIST Classificaion

Name:

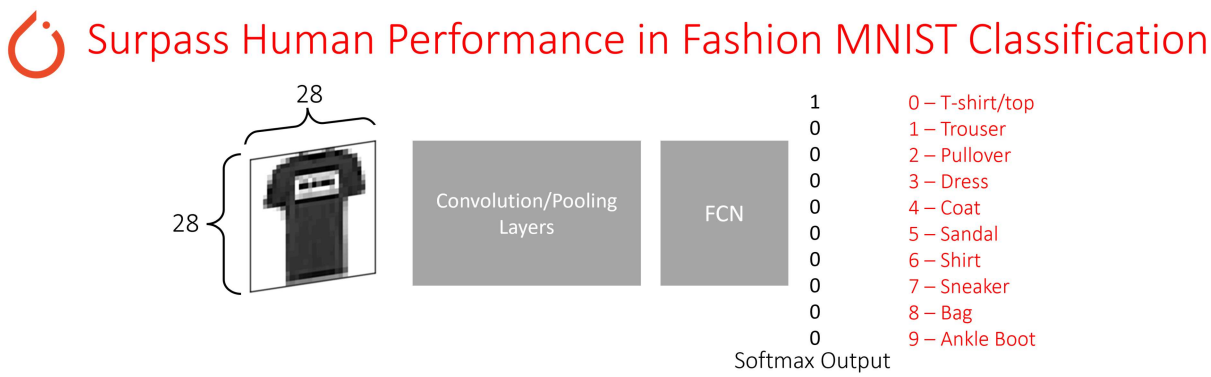
```
%matplotlib inline
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
import numpy as np
```

```
from IPython.display import Image # For displaying images in colab jupyter cell
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
Image('/content/drive/MyDrive/lab4_exercise.png', width = 1000)
```



Prepare Data

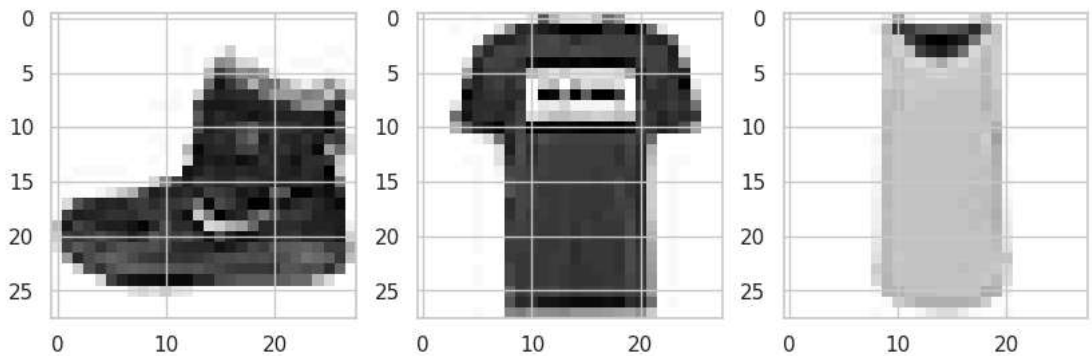
```
# Load Fashion-MNIST Dataset in Numpy

# 10000 training features/targets where each feature is a greyscale image with shape (28, 28)
train_features = np.load('/content/drive/MyDrive/fashion_mnist_train_features.npy')
train_targets = np.load('/content/drive/MyDrive/fashion_mnist_train_targets.npy')

# 1000 testing features/targets
test_features = np.load('/content/drive/MyDrive/fashion_mnist_test_features.npy')
test_targets = np.load('/content/drive/MyDrive/fashion_mnist_test_targets.npy')

# Let's see the shapes of training/testing datasets
```

```
print("Training Features Shape: ", train_features.shape)
print("Training Targets Shape: ", train_targets.shape)
print("Testing Features Shape: ", test_features.shape)
print("Testing Targets Shape: ", test_targets.shape)
```

```
Training Features Shape:  (10000, 28, 28)
Training Targets Shape:  (10000,)
Testing Features Shape:  (1000, 28, 28)
Testing Targets Shape:  (1000,)
```

```
# Visualizing the first three training features (samples)

plt.figure(figsize = (10, 10))

plt.subplot(1,3,1)
plt.imshow(train_features[0], cmap = 'Greys')

plt.subplot(1,3,2)
plt.imshow(train_features[1], cmap = 'Greys')

plt.subplot(1,3,3)
plt.imshow(train_features[2], cmap = 'Greys')
```

```
<matplotlib.image.AxesImage at 0x7dfc766749e0>
```




```
# Define your scaling function
def scale(array):
  return (array-np.mean(array))/np.std(array)

# Scale the dataset according to standard scaling
train_features = (train_features-np.mean(train_features))/np.std(train_features)
test_features = (scale(test_features)-np.mean(train_features))/np.std(train_features) #scale according to how we scaled the tra
```

```
# Take the first 1000 (or randomly select 1000) training features and targets as validation set

validation_features = train_features[0:1000,:,:]
validation_targets = train_targets[0:1000]

# Take the remaining 9000 training features and targets as training set

train_features = train_features[1000:10000,:,:]
train_targets = train_targets[1000:10000]
np.size(train_features)
```

```
7056000
```

```
# Reshape train/validation/test sets to conform to PyTorch's (N, Channels, Height, Width) standard for CNNs

train_features = np.reshape(train_features,(9000,1,28,28))
validation_features = np.reshape(validation_features,(1000,1,28,28))
test_features = np.reshape(test_features,(1000,1,28,28))
```

Define Model

```
# Define your CNN architecture here

class CNNModel(torch.nn.Module):
```

```
    def __init__(self):

        super(CNNModel, self).__init__()

        self.Clayer_1=nn.Conv2d(1,16,3,stride=1,padding=1) #first convolutional layer
        self.bn1 = nn.BatchNorm2d(16) #batchnorm the layer
        #dim_=(dim-3+2*1)/1+1
        self.pool1=nn.MaxPool2d(2) #pool the layer
        #dim=dim//2+1
        self.Clayer_2=nn.Conv2d(16,32,3,stride=1,padding=1) #second convolutional layer
        self.bn2 = nn.BatchNorm2d(32) #batchnorm the layer
        #dim=(dim-3+2*1)/1+1
        self.pool2=nn.MaxPool2d(2) #pool the layer
        #dim=dim//2+1

        self.Flayer=nn.Linear(32*7*7,64) #first fully connected layer, input is 64*dim*dim=64*7*7
        self.layer_out=nn.Linear(64,10) #output fully connected layer

        self.act=nn.ReLU() #activation we apply after every layer except the output
        self.dropout=nn.Dropout(p=.2) #dropout we apply after the first fully connected layer

    def forward(self, input):

        #layer 1 (conv.)
        x=self.Clayer_1(input)
        x=self.bn1(x)
        x=self.act(x)
        x=self.pool1(x)

        #layer 2 (conv.)
        x=self.Clayer_2(x)
        x=self.bn2(x)
        x=self.act(x)
        x=self.pool2(x)

        #flatten to make it possible to put in fully conneccted layer
        x=x.view(x.size(0),-1)

        #fully connected layer
        x=self.Flayer(x)
        x=self.act(x)
        x=self.dropout(x)

        #output layer
        output=self.layer_out(x)

        return output
```

Select Hyperparameters

```
# Fix the random seed so that model performance is reproducible
torch.manual_seed(55)

# Initialize your CNN model

model = CNNModel()

# Define learning rate, epoch and batchsize for mini-batch gradient

learning_rate = .0001
epochs = 80
batchsize = 64

# Define loss function and optimizer

loss_func = torch.nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(),lr=learning_rate)

# Run this line if you have PyTorch GPU version
if torch.cuda.is_available():
    model.cuda()
```

Identify Tracked Values

```
# Placeholders for training loss and validation accuracy during training
# Training loss should be tracked for each iteration (1 iteration -> single forward pass to the network)
# Validation accuracy should be evaluated every 'Epoch' (1 epoch -> full training dataset)
# If using batch gradient, 1 iteration = 1 epoch

train_loss_list = []
validation_accuracy_list = []
```

Train Model

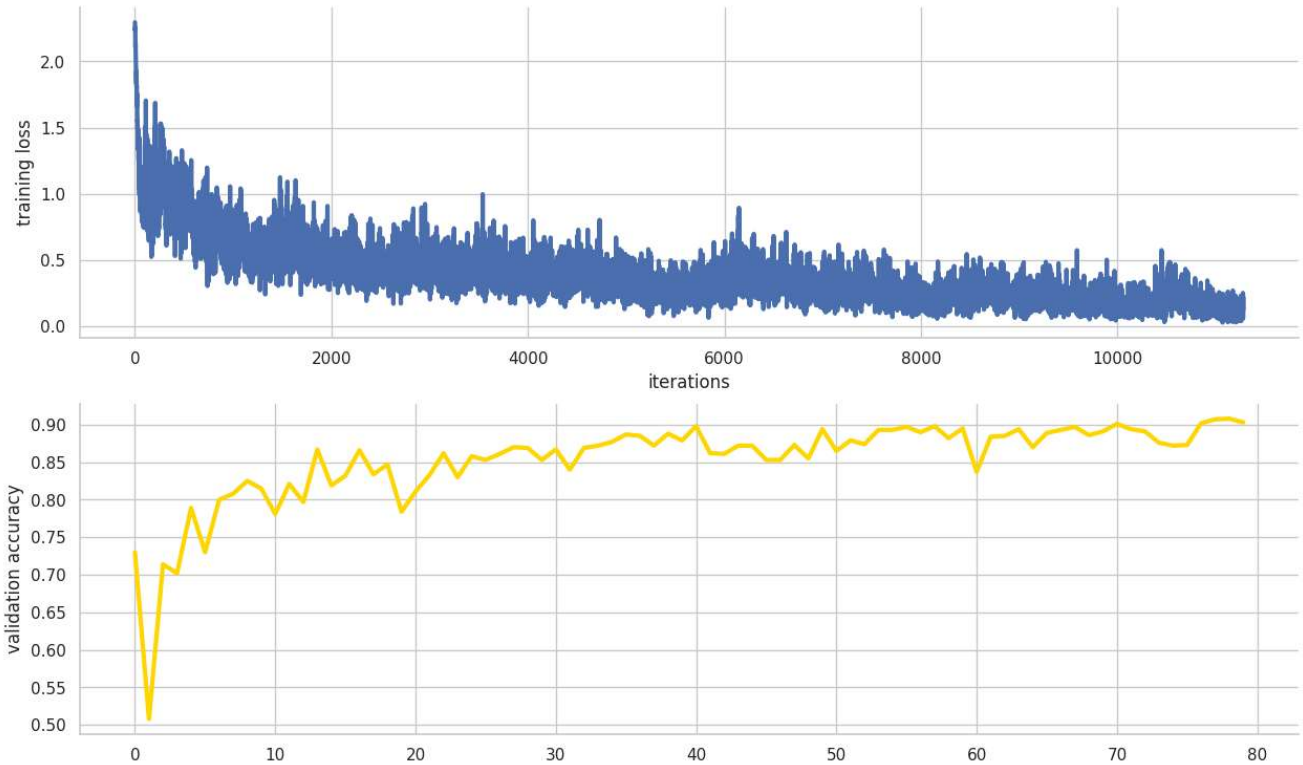
```
import tqdm # Use "for epoch in tqdm.trange(epochs):" to see the progress bar

# Convert the training, validation, testing dataset (NumPy arrays) into torch tensors
# Split your training features/targets into mini-batches if using mini-batch gradient

#make sure they are all in cuda to make it go faaaast :)
train_features=torch.from_numpy(train_features).float().cuda()
test_features=torch.from_numpy(test_features).float().cuda()
validation_features=torch.from_numpy(validation_features).float().cuda()

train_targets=torch.from_numpy(train_targets).long().cuda()
test_targets=torch.from_numpy(test_targets).long().cuda()
validation_targets=torch.from_numpy(validation_targets).long().cuda()

#make training batches
dataset=TensorDataset(train_features,train_targets)
training_loader=DataLoader(dataset,batch_size=batchsize,shuffle=True)

# Training Loop ------------------------------------------------------------------------------------------

for epoch in tqdm.trange(epochs):

    # Set model into training mode
    model.train()

    # Reset gradients, Calculate training loss on model
    optimizer.zero_grad()
    for data, labels in training_loader:
      #compute the model output
      outputs=model(data)
      #compute the loss
      loss=loss_func(outputs,labels)
      #add the loss to the list of losses per iteration
      train_loss_list.append(loss.item())

      # Perfrom optimization, back propagation
      loss.backward()
      optimizer.step()

    # Telling PyTorch we aren't passing inputs to network for training purpose
    with torch.no_grad():
        model.eval()

        # Compute validation outputs (targets)
        val_outputs=model(validation_features)

        # and compute accuracy
        val_acc=(val_outputs.argmax(dim=1)==validation_targets).sum().item()/validation_targets.numel()
        validation_accuracy_list.append(val_acc) #add the validation accuracy to the list of validation accuracies per epoch
```

```
100%|██████████| 80/80 [00:34<00:00,  2.31it/s]
```

Visualize & Evaluate Model

```
# Seaborn for prettier plot

import seaborn as sns

sns.set(style = 'whitegrid', font_scale = 1)
```

```
# Visualize training loss

plt.figure(figsize = (15, 9))

plt.subplot(2, 1, 1)
plt.plot(train_loss_list, linewidth = 3)
plt.ylabel("training loss")
plt.xlabel("iterations")
sns.despine()

plt.subplot(2, 1, 2)
plt.plot(validation_accuracy_list, linewidth = 3, color = 'gold')
plt.ylabel("validation accuracy")
sns.despine()
```




```
# Compute the testing accuracy

# Compute the testing accuracy

test_accuracy_list=[]

# Telling PyTorch we aren't passing inputs to network for training purpose
with torch.no_grad():

    model.eval()

    # Compute test outputs (targets)
    test_outputs=model(test_features)

    # and compute accuracy
```

```
    test_acc=(test_outputs.argmax(dim=1)==test_targets).float().sum().item()/test_targets.numel()

    test_accuracy_list.append(test_acc)


# Compute total (mean) accuracy
test_accuracy=np.mean(test_accuracy_list)

# Report total (mean) accuracy, can also compute std based on batches
print("Total Accuracy:" + str(test_accuracy*100) + "%")
```

```
Total Accuracy:89.3%
```

```
# (OPTIONAL) Print the testing accuracy for each fashion class. Your code should produce something that looks like:
# Clever usage of np.where() could be useful here

# "Accuracy of T-shirt/top: 93.5 %"
# "Accuracy of Trouser: 89.3 %"
# etc...

# What's the fashion item that your model had the hardest time classifying?
categories = ["T-shirt/top","Trouser","Pullover","Dress","Coat","Sandal","Shirt","Sneaker","Bag","Ankle boot"] #list the categor

predictions=(test_outputs.argmax(dim=1)) #get the predictions
per_cat_accuracies=np.zeros(10) #initialize accuracy list

for i in range(10): #loop of categories
    index=(test_targets == i) #get the indices of the testing samples in a given category
    per_cat_accuracies[i]=(predictions[index] == test_targets[index]).float().mean().item() #calculate accuracy in a given categ
    print("Accuracy of "+categories[i]+": "+str(per_cat_accuracies[i]*100)+"%") #print result

print("Worst per-category accuracy is for "+categories[per_cat_accuracies.argmin()]+".") #pick out lowest accuracy category and
```

```
Accuracy of T-shirt/top: 89.71962332725525%
Accuracy of Trouser: 99.04762506484985%
Accuracy of Pullover: 88.28828930854797%
Accuracy of Dress: 88.17204236984253%
Accuracy of Coat: 78.2608687877655%
Accuracy of Sandal: 97.7011501789093%
Accuracy of Shirt: 70.10309100151062%
Accuracy of Sneaker: 97.8947401046753%
Accuracy of Bag: 91.57894849777222%
Accuracy of Ankle boot: 94.73684430122375%
Worst per-category accuracy is for Shirt.
```